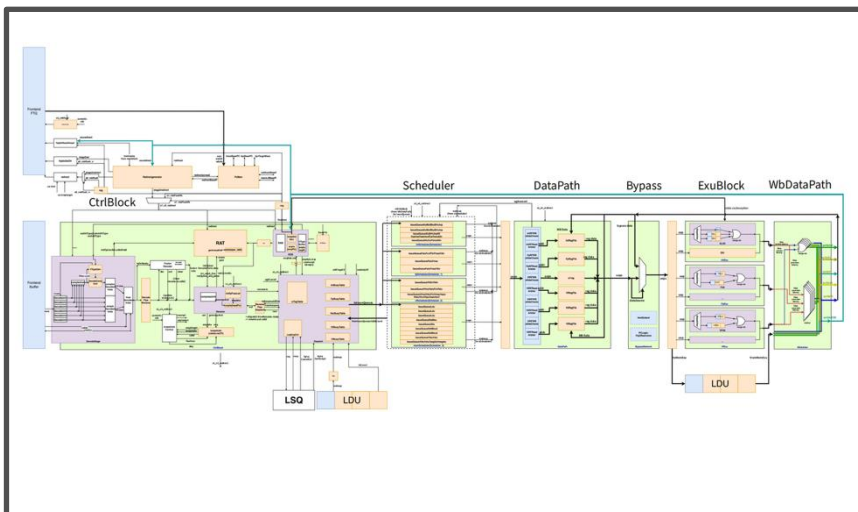


XiangShanLab

香山处理器学习与实践平台 — 学习指引

仓库地址: <https://github.com/OpenXiangShan/XiangshanLab>

官方网站: openxiangshan.cc



Kunminghu V2 后端架构模型

写在前面

一些开发者的话

这份PPT的目的，是为想要了解香山、对香山方向感兴趣的伙伴提供一份指引——帮助你借助 XiangShanLab，一步步找到学习香山的切入点，建立自己的学习节奏。

作为课程开发人员之一，说实话，目前整套课程还远称不上完美，某些地方可能存在错误，部分知识也仍有空白。非常感谢你愿意在课程尚不完善的阶段就给予支持，真的十分感激！

如果在学习过程中遇到任何问题，欢迎随时通过 GitHub Issue 提出批评指正：

<https://github.com/OpenXiangShan/XiangShanLab/issues>

面向三类学习者

基于本课程首次面向学习者的情况，我们将这群学习群体大致分为三类（当然，本课程面向的受众不限于这三类）。因此，本PPT将重点从这三个角度，讲解如何学习本课程。

香山AI方向

教你在熟悉香山架构后，可尝试为其定制并添加面向 AI 加速的自定义指令，实现特定场景的性能优化。

DSU 方向

可以重点看看 Chisel 和 Diplomacy 相关的实战项目，通过动手练，带你入门 SoC 集成和总线互联相关的内容。

香山处理器方向

能学的可多了——这个课程基本把香山从环境搭建到微架构分析、从 RTL 设计到波形调试、从场景追踪到 Bug 学习定位全串起来了，跟着走，相信你能领悟到不少东西。

XiangShanLab仓库介绍

本课程目前暂以 GitHub 仓库为载体，所有资料均托管于：

<https://github.com/OpenXiangShan/XiangShanLab>

教学文档几乎全部采用 Markdown 格式，可克隆至本地学习查阅。

下面首先为大家简要介绍该仓库的主要内容。

项目根目录下有以下几个目录：

xiangshan-course 课程文档

不难看出，此目录下的就是**本课程最核心的内容**由**八个章节**，构建了完整的知识体系。无论你属于哪类学习者，都需依托这一体系展开学习。

xiangshan-programming-practice 编程实践

本目录中存放的是一些**编程实践**，这些代码已经基本完成（当然，这不意味着完美，欢迎批评指正），包括简易非阻塞Cache、Diplomacy实战项目等。

xiangshan-question-bank 题库

就是个题库，但经过系统组织，划分为9个体系类别。题目来源扎实：经典问题、开芯院笔试题、实际工作真实场景，无一凭空而来。**部分题目仍待完善，期待你来解答！**

tools 工具

在AI发展迅猛的当下，学会使用AI依旧是一项重要技能。下面放置的工具也是我们学习开发课程时使用到的Skill工具，相信对你的学习也一定有所帮助。

2026-CIE-RISC-V-Contest-Application-Track

CIE电子大赛

赛题提交区：**为香山添加自定义指令（本课程组命题）** 参赛即获奖，参赛周期长，欢迎踊跃报名。

<https://rv.cie.org.cn/direction?id=8>

香山课程体系详细介绍

整个课程的**灵魂与精髓**都集中在 **xiangshan-course 目录**下，共包含八个章节。下面为大家简要介绍一下。

1-xiangshan-development-environment 开发环境

本章是零基础学习者的友好入门章节。**无需任何前置基础，只需跟随教程操作，即可成功运行香山**，并掌握环境搭建方法。其中，**Chapter 1 HelloXiangShan 就是一个特色任务**，非常欢迎零基础同学从这里起步。

当然，如果你想进一步了解 NEMU、Spike、Difftest 等知识，同样可以在本章找到相关解答。

3-riscv-specification

RISC-V规范

主要析指令集架构规范，对RISC-V指令集不熟悉的同学可以来看看

5-xiangshan-scenarios-analysis

场景分析

本课程的重中之重！这也是区别于其他纯理论书籍的最大特色——不枯燥地空谈超标量理论，而是真正带你深入香山的**运行过程**，在**动态执行**中领悟微架构的设计精髓，在**具体场景**中看懂香山到底是怎么做的。

2-xiangshan-programming 香山编程

主要讲解 Chisel 与 Diplomacy 的编程基础。如果你有 Verilog 基础，学起来会相对轻松。当然，这里并非枯燥的理论堆砌，而是穿插了实战任务，帮助你在实践中加深理解。

4-xiangshan-microarchitecture-analysis

微结构分析

真正开始学习香山的微架构的起步的地方。这部分是课程的精髓之一——对香山微架构的剖析包含大量理论知识：从最基础的超标量理论讲起，再过渡到香山中的具体实现方案，同时也逐步展开代码解析。

6-xiangshan-debug

香山调试

这其实是本课程的动机之一。课程的目标，是让学员**具备开发香山的能力**——而在所有开发能力中，最核心的其实是 debug 能力。因此，本章节会带大家回顾香山历史开发中遇到的一些 bug，让你在分析和解决问题的过程中，逐步领悟如何真正具备**开发香山**的能力。

7-xiangshan-development 与 8-xiangshan-development-tools 是后续讲解真正开发的教程，目前课程也正在持续开发中。

第一部分：公共基础部分

针对本课程首次面向的三类学习者——香山AI方向、DSU方向、香山处理器方向，

因此本次教程偏向为这三类人群讲解学习思路与路径。

所以首先讲解建议这三类人群在开始学习时都必须经过的基础部分。

如果你是从未接触过香山的纯新手

通关时间：1 day

如果你之前从未接触过香山，比如只知道它的 GitHub 网址，不需要焦虑要学习什么前置知识才能真正开始玩香山，在不学习任何前置知识的情况下，完全可以立刻开始我们的经典任务：**Hello XiangShan**。

任务地址：

https://github.com/OpenXiangShan/XiangShanLab/blob/master/xiangshan-course/docs/1-xiangshan-development-environment/Chapter_1_Overview_of_the_Xiangshan_Development_Environment.md

学习效果：顺利的话，香山会在你的指引下打印出任意你想要的文本。完成此任务后，恭喜你——**已具备运行香山的能力**，后续学习你将以相同的方式反复运行它，也能让它执行任何你指定的程序。

当然，香山由人开发，出错在所难免。如何及时发现问题？同级目录下提供了丰富的 NEMU、Spike、Difftest 相关资料，带你认识香山开发中不可或缺的验证工具链，值得花时间了解。

```
[PERF ][time=4505] SimTop.0
[PERF ][time=4505] SimTop.1
zzx@zzx:~/xiangshan/xs-env/XiangShan$ ./bui
emu compiled at Jul 14 2026, 00:07:05
Using simulated 32768B flash
Using simulated 8386560MB RAM
The image is /home/zzx/xiangshan/xs-env/nex
Hello, XiangShan!
```

提示一下：你要是能搞定这个任务，就已经有通关 CIE全国RISC-V高水平创新及应用大赛 初级阶段的能力了。欢迎报名参赛！**万一参加的人很少，光靠这个任务就能拿奖呢~**

链接：<https://rv.cie.org.cn/direction?id=8>

如果你之前从未接触过Chisel

通关时间：5 天（因个人差异会有所不同）

香山采用 Chisel 开发，学习 Chisel 自然是理所应当的。得益于 AI 的快速发展，如今学习一门新语言的效率极高，但仍建议至少先过一遍基础内容，可参照以下地址中的学习资料。

任务地址：

<https://github.com/OpenXiangShan/XiangShanLab/tree/master/xiangshan-course/docs/2-xiangshan-programming/1-chisel>

学习效果：不要求熟练掌握 Chisel 高度抽象的语法特性，但应在短期内认识常见 Chisel 用法，能够使用纯 Chisel 完成简单的 RTL 设计与验证——例如达到类似 NonBlockingCache 项目（<https://github.com/OpenXiangShan/XiangShanLab/tree/master/xiangshan-programming-practice/NonBlockingCache>）的水平。刚开始写 "Verilog 风格" 的 Chisel 代码也是完全允许的。

检验你是否真正掌握的最佳方式，就是**动手实践**。如果你还有充裕的时间巩固基础，不妨尝试用 Chisel 完成一些小项目——题库中（<https://github.com/OpenXiangShan/XiangShanLab/blob/master/xiangshan-question-bank/2-xiangshan-chisel-programming/chisel-question.md>）有大量题目等你来挑战，例如 Non-blocking Cache 等。部分题目目前尚无解答，如果你能完成，欢迎提交 PR，供后续学习者参考！届时，你的名字也将出现在 XiangShanLab 的 Contributors 中！

关于 RISC-V 指令集

可选，约 3 天（按需）（因个人差异会有所不同）

无论你最终选择哪个方向，既然是做香山相关的东西，了解一下底层的指令集架构（ISA）总归是有益无害的。

这部分内容围绕 RISC-V 规范展开，帮你把指令、寄存器、特权级这些基础概念理清。不过话说回来，这一步不是强制要求——如果你更想快点动手，完全可以先跳过，等后面遇到具体指令时再回头补。

学习效果：学完之后，你至少应该能看懂一条 RISC-V 指令到底在干什么，知道常用的整数、乘法、原子指令大致是怎么回事，知道 riscv 指令集中包含了哪些指令，为后面理解香山的取指、译码、执行打下一个底子。

用到的时候知道去哪查就够了。

任务地址：

github.com/OpenXiangShan/XiangShanLab/tree/master/xiangshan-course/docs/3-riscv-specification

对于 AI 和 DSU 方向的同学，如果时间紧张，这章可以先放一放；AI 方向的同学可以重点了解一下向量指令集相关的内容。

但如果你是香山处理器方向，还是建议认真过一遍，后面分析微架构时会经常回来翻

我们还有一个 RISC-V 规范的学习 Skill 工具

<https://github.com/OpenXiangShan/XiangShanLab/tree/master/tools/riscv-skill-pack>

必做：系统地认识一下 香山微架构

约 5 天（按需）（因个人差异会有所不同）

不管你走哪个方向，这一步 建议所有人都认真看看。毕竟咱们折腾的都是香山，先对它的整体架构有个清晰的认知，后面的路才好走。

这章从最基础的 **超标量理论**讲起，再一层层过渡到香山里具体的 **流水线**、**分支预测**、**缓存** 等模块。

目标是让你 对香山长什么样心里有数，后面无论是改代码还是调 bug，脑子里能有一个完整的地图。

学习效果： 学完这章，配合题库练习，你应该能 **比较扎实地答出**仓库里 **xiangshan-question-bank/5-xiangshan-microarchitecture** 中的大部分题目——这算是一个检验自己是否真的学进去了的好办法。

任务地址：

github.com/OpenXiangShan/XiangShanLab/tree/master/xiangshan-course/docs/4-xiangshan-microarchitecture-analysis

这章理论不少，别想着一口气啃完，**每天看一点、配合着后面场景分析的实操来回对照**，理解会扎实很多。

另外，在tool工具下还有很多适合在这一步来使用的skill工具，例如 xiangshan-code-analyzer，配合着AI工具的使用来提高学习的效率

第二部分：各个方向的建议

针对本课程首次面向的三类学习者——香山AI方向、DSU方向、香山处理器方向，

因此本次教程偏向为这三类人群讲解学习思路与路径。

接下来着重介绍建议这三类人群在开始学习时可以走的路径。

AI方向：先搞懂香山是怎么执行一条简单指令的

通关时间：1 天（之后反复回顾、可学习更复杂的指令）

想加指令，就得先知道香山 到底是怎么跑一条指令的。这就要用到仓库里特有的 场景分析（scenarios-analysis） 章节。

它最大的特点就是 不空谈理论，而是带着你 在真实运行过程里看微架构——一条指令从译码、发射到写回，每一步在香山里是怎么流动的，都给你扒得明明白白。先花一天时间，把最基础的加法（add）的执行过程吃透，先把整体流程跑通。再可以去看其他更复杂的其他类别的指令。

学习效果： 当你能在脑子里完整复现一条 add 指令在香山流水线里的生命周期，知道它经过哪些模块、在哪一步可能被阻塞，后面无论是加新指令还是定位 bug，心里就都有底了。

任务地址：

github.com/OpenXiangShan/XiangShanLab/tree/master/xiangshan-course/docs/5-xiangshan-scenarios-analysis

这边建议实操起来，自己运行一些程序、在自己运行的波形中去学习，将波形图和教程文档一起看，边跑仿真边对照教程，比干看文字效率高得多。说不定你还能在你自己运行的新的波形中发现新的知识点！

重点关注： 向量指令的执行过程

通关时间：1周左右

对 AI 方向的人来说，标量加法只是热身，向量（Vector）才是重头戏——毕竟 AI 计算很大程度上就是大量规整的向量/矩阵运算。所以要重点啃 `vector-add` 这类向量指令的执行过程，理解香山里向量寄存器、向量执行单元是怎么配合工作的。不过得提前打个招呼：这部分内容目前仍在开发中，材料不太完善，正好可以自己动手补充。

学习效果：搞清楚 一条向量指令从拆包、分发到并行的完整执行链路，你就抓住了 AI 加速在处理器层面的核心机制，后面设计自定义 AI 指令时，才知道该怎么复用、怎么优化。

任务地址：

github.com/OpenXiangShan/XiangShanLab/tree/master/xiangshan-course/docs/5-xiangshan-scenarios-analysis/scenarios-analysis/instructions-lifecycle/vector-add

章节仍在完善中，遇到内容缺失或错误，欢迎直接在仓库提 Issue 甚至提交 PR 一起补全，也算是为课程做贡献啰。

AI 方向：从 CIE 大赛 入手

通过时间：断断续续，一个月左右（因个人差异会有所不同）

对 AI 方向的同学，目前课程能提供的核心支持就是 CIE 全国 RISC-V 创新大赛 的题目——引导大家给香山添加一条自定义指令。

别被这个题目给吓到，其实就是 围绕一条新指令，把从编译、译码到执行的整条链路打通，这个过程会逼着你把前面学的东西全都串起来。至于具体怎么加、从哪下手，就需要先对香山的执行过程有足够了解——这正是前面两页所要求的

学习效果：顺利完成的话，你就真正做到了为香山扩展指令集——这可不是书本练习，而是实打实能给开源项目做贡献的硬本领，完成度高的话，欢迎提交 PR，你的名字也会出现在 XiangShanLab 的贡献者名单里！

任务地址：

<https://rv.cie.org.cn/direction?id=8>

这个过程注定不会一帆风顺，建议先用本学习仓库中的内容打牢基础，卡住的时候善用 Issue 提问、善用 仓库中的 AI Skill 工具。

万事俱备，开始 **动手开发** 吧

通过时间：断断续续，一个月左右（因个人差异会有所不同）

到这里，**环境搭建**、**Chisel 写法**、**微架构理论**、**执行过程**这些公共知识你都已经有了，终于可以结合前面的积累，开始摸索着做实际开发了。

从改一条简单指令开始，**一步步往复杂了做**——遇到不会的就回头翻教程、看代码、问人，循环往复。另外，仓库的 **7-xiangshan-development**（**真正开发的教程**）目前也还在开发中，后续章节

上线后参考价值会更大，敬请期待。

学习效果：能独立为香山添加并验证一条自定义指令，走过**编码 → 编译 → 仿真 → 调试 → 验证**的完整闭环，这基本就意味着你已经具备参与香山开发的能力了，恭喜！

任务地址：

<https://rv.cie.org.cn/direction?id=8>

开发从来不是线性推进的，卡壳、返工都是常态，保持耐心、善用工具，慢慢就会越来越顺。

DSU 方向：先搞定 Diplomacy 基础

通关时间：约1天

对 DSU 方向的同学，学完公共知识后就可以正式切入 Diplomacy 相关的内容了。Diplomacy是建立在 Chisel 之上的 参数协商与总线互联 框架，也是做 SoC 集成的核心技能。这部分概念比较抽象——什么 参数协商（parameter negotiation）、节点（node）、边（edge），一开始容易一头雾水，很正常。建议 花上三四天时间，把基础概念和教程里的例子啃下来，慢一点没关系，关键是吃透精髓。

学习效果：学完之后，你应该**理解 Diplomacy 里参数如何在模块间自动协商与传递**，看懂一个简单的总线互联是怎么搭起来的，脑子里有个大致的框架，后面动手才不会盲目。

任务地址：

github.com/OpenXiangShan/XiangShanLab/tree/master/xiangshan-course/docs/2-xiangshan-programming/2-diplomacy

Diplomacy 的精髓在于用参数把复杂连接抽象化，**多画图、多跑例子** 会比硬读文档有效得多。

动手实践：Chapter 13 编程练习

通关时间：约 1-2 天

光看概念不够，Chapter 13 就是专门让你动手练的。教程会带你 从零搭建一个 Diplomacy 环境，把前面抽象的概念落到代码上。

经典例子是 搭一个加法器，但别止于照抄——试着自己实现几个参数协商的小例子，比如改改位宽、加个新节点，真正体会协商的过程。这章估计也得 花个两三天，要边学边调试。

学习效果： 能独立搭建 Diplomacy 开发环境，并用它实现若干参数协商的简单例子，说明你真的理解 Diplomacy 的精髓了。

任务地址：

github.com/OpenXiangShan/XiangShanLab/blob/master/xiangshan-course/docs/2-xiangshan-programming/2-diplomacy/Chapter_13_Programming_Practices_with_Diplomacy.md

合上教程，能自己从零写出一个带参数协商的小模块，就算过关。

进阶实战：Chapter 14 + 题库练习

通关时间：约 1-2 周

对 Diplomacy 有了感觉之后，就可以解锁 更高级的用法 了——Chapter 14 的实战练习 会带你做更复杂的参数化设计。遇到卡壳的地方，配套 diplomacy-question 题库 就是最好的练兵场，边学边练、以练促学。这两章加一起，大概再花一两周 会比较从容。

任务地址：

github.com/OpenXiangShan/XiangShanLab/blob/master/xiangshan-course/docs/2-xiangshan-programming/2-diplomacy/Chapter_14_Diplomacy_Practical_Exercise_1.md

对应题库：

github.com/OpenXiangShan/XiangShanLab/blob/master/xiangshan-question-bank/3-xiangshan-diplomacy/diplomacy-question.md

学习效果：

做完之后，你应该能熟练运用 Diplomacy 做中等规模的参数化模块设计，面对一个陌生的 Diplomacy 代码库也能大致看懂、改得动。

实在没有头绪的话，先看答案再自己重写一遍，效果比死磕好得多。

香山处理器方向：路径可复用，再加一项 Bug 学习

通关时间：穿插在全流程中，反复学

对香山处理器方向的同学，完全可以先走和 AI 方向一样的路径：

先通过场景分析搞懂动态执行过程，再尝试参加 CIE 比赛给香山加一条指令——这套路径是可以直接复用的，不用另起炉灶。

但更重要的是，你们还可以额外补上整个课程最精髓的一块：Bug 学习。6-xiangshan-debug 章节收录了香山历史开发中真实遇到过的 bug。通过分析这些真实的、曾经难倒过开发者的 bug，你会学到书本上根本学不到的东西——这恰恰是成为成熟开发者最快的捷径。

学习效果：

不仅能复用 AI 方向的完整开发路径，更能通过分析历史bug 建立起扎实的调试思维——说白了，调试能力才是开发香山最核心的能力，这一章值得反复品味。

任务地址：

github.com/OpenXiangShan/XiangShanLab/tree/master/xiangshan-course/docs/6-xiangshan-debug

本课程的动机其实就是让他的学习者真正能够有开发香山的能力，但做过开发的同学都知道，只有真正拥有 Debug能力 才能称得上是有开发能力的，所以说这个章节是整个课程的重中之重，虽然目前暂不完善，但后续会着重开始优化课程的实。可以边学边调、边学边看历史中出现了哪些bug、越早动手调 bug 越好。

别忘了用起来：仓库里的 AI Skill 工具

通关时间：穿插在全流程中，反复学

最后想特别提醒大家一件事——tools 目录里藏着的这些 AI Skill 工具，能在开发和学习的各个环节极大地提高效率，强烈建议大家用起来！

它们覆盖面很广：

波形自动分析 Skill 可以帮你快速读懂复杂的仿真波形

指令执行过程分析 Skill 可以辅助你查清楚一条指令是怎么跑的，一目了然

情景分析 Skill，结合场景分析章节，理解更深入

代码分析 / Bug 分析 / 指令集手册分析 Skill，读代码、查 bug、翻手册都好用

学习效果：

这些工具就像随时待命的贴心助教，能在你卡壳的时候给思路、在你疲惫的时候省时间。

不管是做开发、调 bug 还是啃新章节，都强烈建议随手用起来——善用工具的人，学习效率会高出一个量级。

但一定要始终对 AI 抱有怀疑态度。

所以，快去试试吧，别让它们躺在仓库里吃灰

任务地址：

github.com/OpenXiangShan/XiangShanLab/tree/master/tools

如果在用的过程中有什么改进想法，也欢迎一起完善这些 Skill，让整个社区的学习体验越来越好！

第三部分：任务领取指南

课程组设计并落地了一套规范化的任务生命周期管理机制（XS-Lab-Status）。课程组通过 Issue 发布任务；学习者可通过标准化的流程认领任务、按规范完成学习或开发任务，并配套积分激励与审查结算。

目前该机制仍处于测试阶段，后续将持续迭代完善。

XS-Lab-Status 任务领取指南

本课程设置了开放式的任务领取机制。无论你来自哪里、基础如何，只要你想锻炼自己，都可以来这里领取任务、大展身手！下面，就为大家详细讲解任务的领取指南

一切操作都通过 Issue 完成。记住一句话：本仓库里没有"聊天群布置任务"，所有事都写在 Issue 里。五步闭环：

<https://github.com/OpenXiangShan/XiangShanLab/issues>

- ① 发起人创建任务 Issue（模板选"任务发布"）
- ② 执行人评论 /assign 认领
- ③ 线下干活（写代码、写文档……）
- ④ 执行人创建 [DELIVERY] 交付 Issue（模板选"任务交付"）
- ⑤ 3 名 Reviewer 各自创建审查 Issue（模板选“审查清单”）

← Create new issue in XiangShanLab/XS-Lab-Status: 任务发布

Add a title *

[TASK]

请完整填写以下内容。执行人评论 /assign 后，由 Issue 发布者回复 /confirm 确认认领；完成后由执行人评论 /complete 发起审查。

任务背景与目标 *

说明为什么需要该任务、预期结果，以及它与整体目标的关系。

Write Preview H B I ≡ <> 🔗 ≡ ≡ ≡ @ 📎 📎 ↶

描述任务背景、目标和范围。

📎 Paste, drop, or click to add files

交付清单 *

每项交付物必须写成可勾选的子项。

Write Preview H B I ≡ <> 🔗 ≡ ≡ ≡ @ 📎 📎 ↶

☐ Create more Cancel Create 🚀

发起人创建任务

XS-Lab-Status 认领任务指南

进入你想做的那个任务 Issue 页面，拉到最底部评论框，输入：

`/assign`

点击 `Comment` 提交。

前提条件：

你当前可用积分 $\geq$ 质押额。每人初始 100 分，质押 = 复杂度 $\times$ 10（比如复杂度 3，质押 30 分）。

效果：

你变成这个 Issue 的负责人（右侧出现你的头像）

标签从 pending 变成进行中

你的积分里冻结对应质押额

任务发布者确认任务示意



做不完怎么办？在任务 Issue 评论框输入 `/extend`，追加 20% 质押（如原质押 30，延期后变 36）。

XS-Lab-Status 任务提交交付

提交交付： 用模板，别裸写

任务做完后，到仓库 Issues 页面 → 点 "New issue" → 选 "任务交付" 模板。

标题改成： [DELIVERY] 关联任务 #编号（编号填对应的任务 Issue 号）

正文里你需要填的关键项：

关联任务： 填任务 Issue 编号和标题

交付清单完成情况：完成的打 [x]，附上证明链接（PR #编号 或 commit 链接）；没完成的写原因，别藏着

交付元数据： 完成度自评（如 80%）、是否延期、实际完成时间、交付物链接

自评说明： 1-5 分给自己打分，写理由

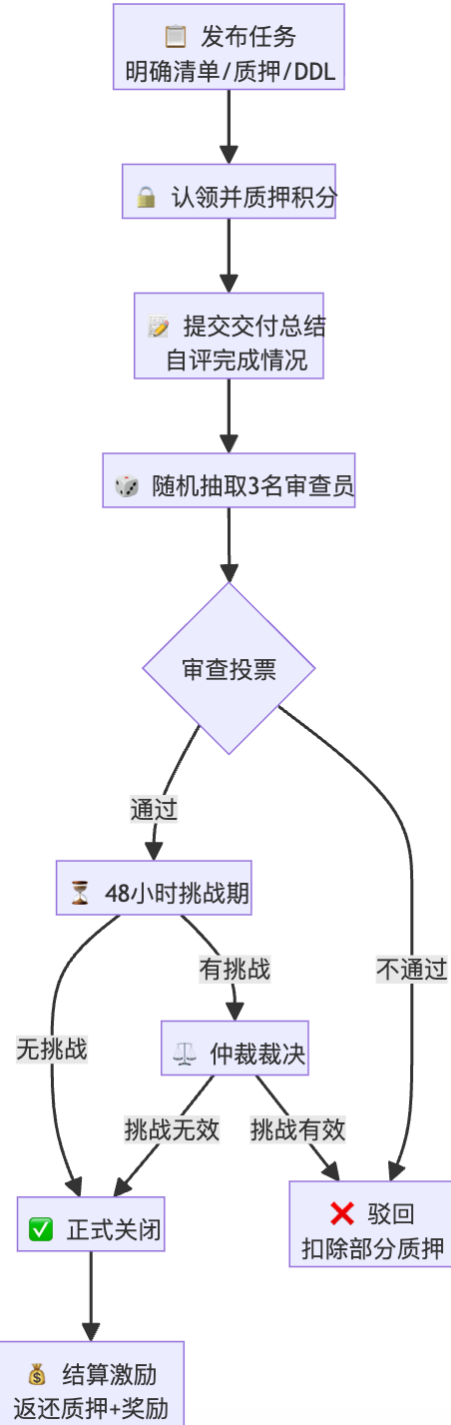
提交后，在交付 Issue 的评论框输入：

/ready-for-review

这就通知 Reviewer 来审你了。

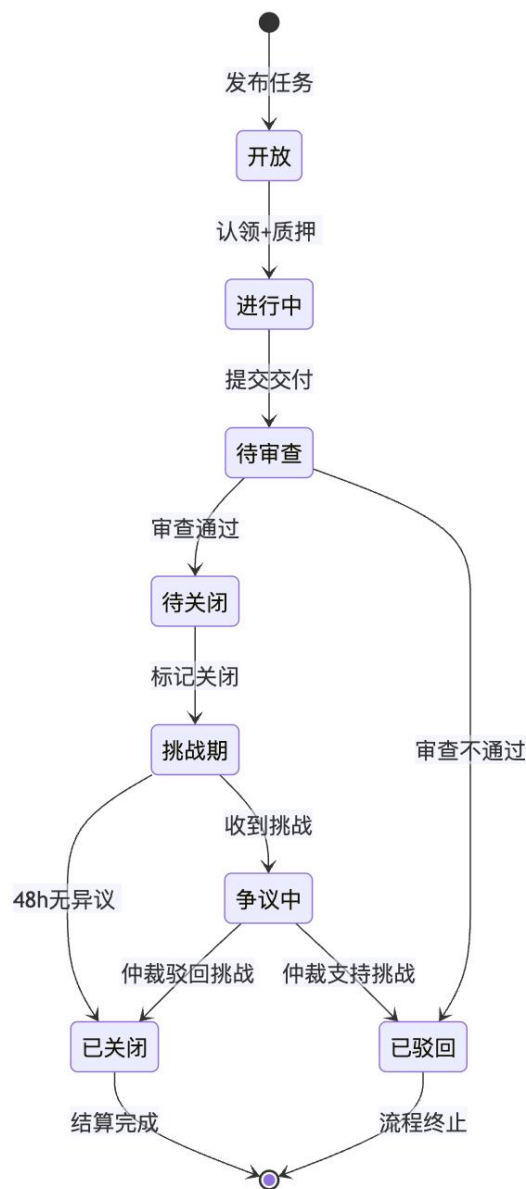
提示：整个系统测试链接为

<https://github.com/OpenXiangShan/XiangShanLab/issues>



← 核心业务流程

↓ 任务生命周期





这份学习路径就先聊到这里。无论你选择哪条方向，重要的是先动起来、持续走下去。

遇到问题别犹豫，善用 Issue、善用 AI Skill 工具。

如果发现课程中的任何问题、或者想要在课程中贡献内容，欢迎随时提出Issue或者提起PR。

期待在 XiangShanLab 的贡献者名单里，看到你的名字 ~

香山课程组

2026年8月

